

Coding & Solution

Date	3 July 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the implementation, quality, and completeness of the **Rising Waters** project. The application is developed using **Python, Flask, HTML, CSS, JavaScript, Bootstrap, and Scikit-learn**. It uses a **Decision Tree Machine Learning model** to predict flood risk based on historical weather and rainfall data.

The solution demonstrates clean coding practices, modular architecture, reusable components, and a responsive user interface. The application successfully addresses the problem of flood prediction by providing accurate predictions and supporting early warning for disaster preparedness.

1. Ensure the application follows a modular Flask project structure with separate folders for templates, static files, and the trained machine learning model.
2. Implement all core functional requirements, including weather data input, flood prediction, result display, and input validation.
3. Include all required source files such as app.py, HTML templates, CSS, JavaScript, the trained **Decision Tree model** (decision_tree_flood.pkl), requirements.txt, and README.md.
4. Maintain clean, well-commented, and reusable code following Python and Flask best practices.
5. Verify that the application runs successfully in a virtual environment using the required dependencies.
6. Include setup instructions for installing dependencies, activating the virtual environment, and running the Flask application.
7. Attach or provide the GitHub repository link containing the complete source code and project documentation.

Solution Summary

Field	Details
Repository Link / URL	[Paste your GitHub / GitLab or hosted link here]
Programming Language(s)	Python, HTML5, CSS3, JavaScript
Framework(s) Used	Flask, Bootstrap 5, Scikit-learn, Pandas, NumPy, Joblib
Key Features Implemented	Home Page, Flood Prediction Form, Decision Tree Model Integration, Flood/No Flood Result Page, Input Validation, Responsive UI, Error Handling
Pending / Incomplete Features	Real-time Weather API Integration, SMS/Email Alerts, Cloud Deployment, Prediction History
Setup / Run Instructions	Create a virtual environment → Install dependencies using pip install -r requirements.txt → Activate the virtual environment → Run python app.py → Open the Flask application in a web browser



Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes

6	Code is committed to a version control repository	
---	---	--

- **Additional Notes / Comments:**

The application uses a **Decision Tree Machine Learning model** with an accuracy of **96.55%** for flood prediction.

- The project follows a modular Flask architecture, making the code easy to understand, maintain, and extend.
- The user interface is responsive and provides an interactive experience for predicting flood risk.
- Future enhancements include integration with live weather APIs, cloud deployment, GIS mapping, and SMS/Email alert services.